

A Project Report on

MEDIportal

A project report Submitted in partial fulfillment of the requirements for the award of

**DIPLOMA IN
COMPUTER ENGINEERING**

Submitted by

B.Purna Chandra Rao
23354-CM-027

Under The Esteemed Guidance Of

Ms.K.JAYA SRI,_{B.Tech}



**DEPARTMENT OF COMPUTER ENGINEERING
D.N.R COLLEGE OF ENGINEERING AND TECHNOLOGY
(Approved by AICTE, New Delhi & Affiliated to SBTET, Mangalagiri)
(Accredited by NAAC with A++ Grade & NBA)
BHIMAVARAM
2025 -2026**

D.N.R. COLLEGE OF ENGINEERING & TECHNOLOGY

(Approved by AICTE, New Delhi & Affiliated to JNTUK, Kakinada)

(Accredited by NAAC with A++ Grade & NBA)

Balusumudi, Bhimavaram -534202 W.G.Dist.,A.P., India.

DEPARTMENT OF COMPUTER ENGINEERING



CERTIFICATE

This is to certify that the project report titled “**MEDIPORTAL** ” is a Bonafide record of project work carried out by, **B.PURNA CHANDRA RAO** bearing with the Regd. No: **23354-CM-027** student of Diploma final year in the Department of Computer Engineering, in D.N.R. COLLEGE OF ENGINEERING & TECHNOLOGY, Balusumudi, Bhimavaram. The project work is carried out during the academic year 2025-2026 in partial fulfillment of the requirements for the award of the degree of Diploma in Computer Engineering.

Guide & Internal Examiner

Head of the Section

External Examiner

ACKNOWLEDGEMENT

We are the students of D.C.M.E. of final year, we have done the esteemed project “**MEDIPORTAL**” expressing thanks to our beloved guide Ms. K.JAYA SRI B.Tech., for her active participation and excellence guidance at every stage and for high dynamic and motivate encouragement in successfully completing this project. We expressing our thanks to all staff of D.C.M.E department for helping us whenever we fell their need. We express our thanks to our lab technicians and also another men who are directly or indirectly involved and without whose help we could not have completed this project. Finally, we are very much great full to our parents who have provided the required financial support in completing this project.

Name of the student
B.PURNA CHANDRA RAO

TABLE OF CONTENTS

ABSTRACT	i
LIST OF FIGURES	ii
KEY WORDS	iii
1. INTRODUCTION	1
2. OBJECTIVES OF THE PROJECT	2
3. EXISTING SYSTEM OVERVIEW	3
4. PROJECT SCOPE	4
5. METHODOLOGY AND TECHNOLOGIES USED	5
5.1. REQUIREMENT ANALYSIS	
5.2. DESIGNING THE USER INTERFACE	
5.3. DEVELOPMENT WITH JAVASCRIPT	
5.4. CLIENT-SIDE STORAGE	
5.5. TESTING AND VALIDATION	
5.6. HTML (HYPERTEXT MARKUP LANGUAGE)	
5.7. CSS (CASCADING STYLE SHEETS)	
5.8. JAVASCRIPT	
5.9. BROWSER / CLIENT-SIDE MEMORY	
6. SYSTEM ANALYSIS	6
6.1. PURPOSE OF THE SYSTEM	
6.2. EXISTING SYSTEM ANALYSIS	
6.3. PROPOSED SYSTEM ANALYSIS	
6.4. FUNCTIONAL ANALYSIS	
6.5. NON-FUNCTIONAL ANALYSIS	
6.6. ADVANTAGES OF THE SYSTEM	
7. SYSTEM DESIGN	7
7.1. DATA FLOW DIAGRAM	7
7.2. USE CASE DIAGRAM	8
7.3. SEQUENCE DIAGRAM	9
7.4. CLASS DIAGRAM	10
7.5. ARCHITECTURE DIAGRAM	11
7.6. UI DESIGN	12
8. IMPLEMENTATION	13-15
9. RESULTS AND OUTPUT SCREENSHOTS	16-18
9.1 LOGIN SCREEN	
9.2 MAIN MENU SCREEN	
9.3 CART SCREEN	
9.4 ORDER SCREEN	

9.5 ORDER TRACKING SCREEN	
10. CHALLENGES FACED	19-20
11. FUTURE SCOPE / MODIFICATIONS	21
12. CONCLUSION	22
13. REFERENCES	23
14. APPENDICES-I	24-44
15. APPENDICES-II	45-47

ABSTRACT

The Online Medicine Delivery System is a web-based application designed to simplify and digitize the process of purchasing medicines from the comfort of home. It offers a user-friendly digital platform where individuals can browse a curated list of available medicines, view detailed descriptions, and seamlessly place orders online. By removing the need to physically visit pharmacies, this system enhances convenience and accessibility particularly for elderly individuals, people with disabilities, or those residing in remote or rural areas with limited access to healthcare facilities. The primary objective of this project is to present a functional prototype using only fundamental web technologies such as HTML, CSS, and JavaScript, deliberately avoiding the use of databases or server-side programming. Instead, all medicine-related data, such as names, prices, and descriptions, are stored in the application itself using static arrays or JavaScript objects. This design choice not only simplifies development but also reduces technical complexity, making the system ideal for educational purposes or small-scale use cases. By operating entirely on the client side, the application eliminates the need for complex server configurations or database setup. It can be launched directly in any modern web browser, making it highly portable and easy to deploy. Users can navigate through various interface components to view medicine categories, inspect individual product details, and add selected items to a virtual cart. While this client-only model does not support persistent storage, real-time inventory management, or secure user authentication, it effectively demonstrates the fundamental operations of an online medicine delivery system. Core features such as dynamic content rendering, interactive cart management, and basic user flow from browsing to checkout are fully implemented, offering a comprehensive overview of how a more robust, full-stack application might function. This project serves as a foundational prototype that can be scaled or enhanced in the future by integrating backend technologies such as PHP, Node.js, or Python, along with database systems like MySQL or MongoDB. Additional features such as user login systems, order tracking, payment gateway integration, and inventory updates could be added to transform the prototype into a production-ready application. In summary, the Online Medicine Delivery System is not only a demonstration of how basic web technologies can be used to build interactive and functional web applications, but also a stepping stone for learners and developers looking to expand their knowledge into more complex areas of web development. It bridges the gap between theoretical understanding and practical implementation, illustrating how digital solutions can improve accessibility and efficiency in the healthcare domain.

LIST OF FIGURES

FIG NO.	DESCRIPTION	PG.NO
7.1	Data Flow Diagram	7
7.2	Use case Diagram	8
7.3	Sequence Diagram	9
7.4	Class Diagram	10
7.5	Architecture Diagram	11
7.6	UI Desisgn	12
9.1	Login Screen	16
9.2	Main Menu Screen	16
9.3	Cart Screen	17
9.4	Order Screen	17
9.5	Order Tracking Screen	18

KEYWORDS

Abbreviation	Expansion
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
UI	User Interface
GUI	Graphical User Interface
AI	Artificial Intelligence
RAM	Random Access Memory
ROM	Read-Only Memory
CPU	Central Processing Unit

CHAPTER-1

INTRODUCTION

The rapid growth of the internet has transformed how essential services are accessed, including the healthcare sector. One such advancement is the concept of online medicine delivery, which enables people to order medicines conveniently from their homes. This system is particularly useful for patients who may have difficulty traveling, people in remote areas, or those who simply prefer digital solutions over physical visits to a pharmacy. The proposed project is a simplified version of such a system, developed using basic web technologies without integrating any database connectivity. This project is built using core web technologies like HTML, CSS, and JavaScript, which provide the structure, design, and interactivity of the application. Instead of connecting to a backend database, medicine information and order details are handled using static data stored within the program itself. This makes the system lightweight, easy to deploy, and accessible through any modern web browser. Although it lacks permanent storage, the project is an effective demonstration of how fundamental technologies can create a functional application. The system allows users to browse through a list of medicines, view their details, and add selected items to a virtual cart. The order details are stored temporarily on the client side using browser features like JavaScript variables or session storage. While this approach does not provide long-term record-keeping, it successfully simulates the process of online ordering and checkout. Thus, the project highlights the essential steps in medicine delivery, from selection to confirmation, without the complexity of database management. Overall, this project serves as a prototype that can be extended in the future by adding database connectivity and backend support for real-time inventory and secure data management. For now, it provides an educational and practical model to understand how online services are built using simple web technologies. By avoiding database integration, the system remains easy to understand, making it an ideal project for learners and small-scale demonstrations of online medicine delivery concepts. Medicines are very important in our daily life, and many times people find it difficult to go outside to buy them, especially when they are sick or live far away from medical shops. To solve this problem, online medicine delivery systems are created. These systems allow people to order medicines through a website from their home. This project shows how such a system can work in a simple way using only web technologies. Through this website, a person can see the list of medicines, check their details, and add them to a cart, just like shopping online. The cart is temporary, meaning it remembers the order only while the website is open. Even though it does not save the order permanently, it clearly shows how online medicine delivery works. This project is a starting point or a sample model. In the future, more advanced features like database connection, login system, and payment gateway can be added to make it more useful. But in its simple form, it helps us learn how online delivery websites work and why they are so helpful for people in real life.

CHAPTER-2

OBJECTIVES OF THE PROJECT

The Online Medicine Delivery System has several key objectives aimed at simplifying access to essential healthcare products while also serving as an educational tool for web development. The first and foremost objective is to provide a simple, user-friendly platform that enables users to order medicines online with ease. For many individuals especially those who are ill, elderly, disabled, or living in remote areas visiting a physical pharmacy can be difficult and time-consuming. This system eliminates that inconvenience by allowing users to view a list of available medicines and place their orders directly through a web browser from the comfort of their homes. Another important objective is to demonstrate how an online medicine delivery platform can be built using only basic web technologies such as HTML, CSS, and JavaScript, without relying on databases or server-side programming. By storing all medicine-related data directly in the code using static arrays or JavaScript objects, the system avoids the complexity of database integration. This makes the project especially valuable for beginners in web development, offering a practical, hands-on example of how client-side technologies can be used to build interactive, functional applications without backend dependencies. A third objective is to simulate the core functionalities of an e-commerce platform in a simplified, easy-to-understand format. The system includes features such as browsing products, adding items to a virtual shopping cart, and placing simulated orders. Although the data is not permanently stored and there is no real-time inventory or user account system, these features effectively mirror the typical flow of a real-world online shopping experience. This allows users and developers to understand the underlying mechanics of online transactions and how web interfaces manage user interactions. Additionally, the system aims to offer a temporary yet fully operational solution that requires no special software, installations, or server setups. Because all data and logic are handled on the client side, the application can run on any modern web browser with no configuration needed. This adds a layer of portability and ease-of-use, making it accessible to a wide range of users and developers. Another objective is to provide a realistic and interactive shopping experience. Users can search for medicines, view descriptions, add products to a cart, and simulate the checkout process all of which replicate the primary steps involved in online purchasing. Even though the cart data is stored temporarily and does not persist beyond the session, it successfully mimics the user flow of a real medicine delivery website, helping users visualize how such systems operate. Lastly, a significant objective is to lay the groundwork for future expansion. While the current version of the project is basic and does not include advanced features such as secure login systems, payment gateways, or persistent order tracking, it is designed with extensibility in mind. Developers can build upon this foundation by integrating backend technologies, database support, user authentication, and more sophisticated order management. This makes the project not just a standalone solution but a scalable prototype capable of evolving into a comprehensive, production-ready online medicine delivery platform. In summary, the Online Medicine Delivery System is a multi-purpose project that combines convenience for users with educational value for developers. It simplifies the medicine ordering process, showcases the power of client-side web technologies, and provides a clear path for future growth all within a lightweight, browser-based application.

CHAPTER-3

EXISTING SYSTEM OVERVIEW

In today's digital age, many people rely on online platforms and mobile applications to purchase medicines conveniently from their homes. These professional systems are highly advanced, incorporating robust features such as search functionality, real-time pricing, availability status, prescription uploads, secure payment gateways, and comprehensive user account management. They are connected to powerful backend servers and databases, which store essential data such as customer profiles, order history, medicine inventory, and delivery tracking information. This seamless integration ensures a smooth user experience, allowing customers to log in, view past purchases, track ongoing orders, and easily reorder medicines all within a secure and efficient environment. However, the development of such systems requires a high level of technical expertise. They depend on complex architectures involving backend programming languages (like PHP, Node.js, or Python), database systems (such as MySQL, PostgreSQL, or MongoDB), and secure server-side operations. These components work together to support real-time stock updates, secure data transactions, user authentication, and more. While such systems are essential for serving large-scale user bases and managing live pharmacy operations, they are not beginner-friendly and can be overwhelming for those just starting to learn web development. In contrast, our project focuses on building a simplified model of an online medicine delivery system using only HTML, CSS, and JavaScript. One of the major limitations of this system is the absence of a backend or database. As a result, it cannot store persistent data like customer details, order history, or cart contents beyond the current session. Once the browser is closed or the page is refreshed, all stored information is lost. Unlike real-world systems, users cannot create accounts, log in, track previous orders, or reorder items based on past activity. Despite these limitations, the project still provides a valuable demonstration of the core workflow of an online medicine delivery system. Users can browse a pre-defined list of medicines, view their descriptions and prices, add items to a virtual shopping cart, and simulate the order placement process. While the experience lacks backend processing, it accurately reflects the fundamental steps of e-commerce interaction, such as product selection, cart management, and checkout simulation. This simple model is particularly useful for educational purposes, serving as a foundation for understanding how online shopping systems work. It helps beginners grasp key concepts such as interface design, user interaction, and client-side data handling using JavaScript. Without the added complexity of server setup or database integration, learners can focus on building logic, improving UI/UX, and understanding how front-end components interact. In contrast, professional-grade systems store and manage vast amounts of structured data using reliable databases. This allows users to access their information across sessions, log in from different devices, receive personalized recommendations, and ensure that medicine availability is always up to date. The security, scalability, and performance of these systems are critical to their success, especially when dealing with sensitive health-related data.

CHAPTER-4

PROJECT SCOPE

The main scope of this project is to develop a basic online medicine delivery system that operates entirely within a web browser environment, without the need for any backend servers or databases. The system allows users to browse a predefined list of medicines, view essential details such as names, descriptions, and prices, and add selected items to a virtual shopping cart. This interactive experience simulates the online medicine ordering process and helps users understand how such platforms function, all without requiring a visit to a physical pharmacy. Since this project does not use any database, all medicine-related data and order information are stored directly in the code using JavaScript objects or arrays. This design makes the system extremely lightweight and easy to run, requiring no additional software, installations, or server configuration. As a result, it becomes an ideal project for beginners who are learning web development, offering a hands-on opportunity to build a working application using just client-side technologies. The system enables users to add medicines to a shopping cart, modify their selections, and simulate placing an order. While the data is stored temporarily and cleared once the session ends or the browser is closed, the overall process effectively mirrors the core functionality of real-world e-commerce systems. Users gain a practical understanding of how products are displayed, how a cart operates, and what happens during the order placement process—all within a simple and accessible interface. Moreover, the project emphasizes the practical use of front-end web technologies such as HTML for structuring content, CSS for designing and styling the interface, and JavaScript for adding interactivity and handling application logic. By working on this project, learners can deepen their understanding of how these technologies work together to create dynamic web applications. They also gain insight into concepts like DOM manipulation, event handling, and temporary data storage using tools like `localStorage` or `sessionStorage`. Beyond serving as a basic demonstration, this project lays the groundwork for future enhancements. It has been designed with scalability in mind, meaning that developers can later add advanced features such as user authentication systems, secure login pages, online payment integration, real-time inventory management, and database connectivity to make the system more robust and production-ready. These additions can transform the simple prototype into a fully functional online medicine delivery platform that mirrors professional healthcare e-commerce solutions. Ultimately, the project is developed with educational intent, aiming to provide a clear, practical, and engaging learning experience for students, hobbyists, and aspiring web developers. It offers a tangible way to explore how technology can be applied in the healthcare and e-commerce sectors, bridging theoretical knowledge with real-world application. By working on this project, learners not only build technical skills but also develop an appreciation for how digital solutions can improve accessibility and convenience in critical areas like medicine delivery.

CHAPTER-5

METHODOLOGY / TECHNOLOGIES USED

5.1. Requirement Analysis – First, the requirements of the system were analyzed. This included identifying the main features, such as viewing available medicines, adding them to a cart, and placing an order. The system was planned to be simple and easy to use for beginners.

5.2. Designing the User Interface – The layout of the website was designed using HTML and CSS. HTML provides the structure for pages, tables, buttons, and forms, while CSS is used to make the design visually appealing with colors, fonts, and spacing. The interface is kept simple so users can navigate easily.

5.3. Development with JavaScript – JavaScript is used to make the system interactive. It handles actions like selecting medicines, adding items to the cart, calculating the total amount, and displaying order summaries. All the medicine information is stored in arrays or objects within the code, eliminating the need for a database.

5.4. Client-Side Storage – Temporary data, such as the items in the cart, is stored in the browser's memory during the session. This allows users to see and manage their orders while the website is open, even without a backend database.

5.5. Testing and Validation – The system is tested in different browsers to ensure proper functionality. Users can view medicines, add them to the cart, and see the order summary. Any bugs are identified and corrected to ensure smooth operation.

5.6. HTML (HyperText Markup Language) – Provides the structure of web pages, such as headings, tables, forms, and buttons.

5.7. CSS (Cascading Style Sheets) – Styles the web pages with colors, fonts, spacing, and layout to make the interface user-friendly.

5.8. JavaScript – Adds interactivity, such as adding medicines to the cart, calculating totals, and dynamically updating content on the page.

5.9. Browser / Client-Side Memory – Stores temporary data during the user session, allowing users to interact with the cart and orders.

CHAPTER-6

SYSTEM ANALYSIS

6.1. Purpose of the System

The main purpose of this system is to provide a simple platform for ordering medicines online without visiting a pharmacy. It is designed for learning and demonstration purposes to show how basic web technologies like HTML, CSS, and JavaScript can create an interactive online system.

6.2. Existing System Analysis

Currently, real online medicine delivery systems are advanced. They use databases to store medicine stock, customer details, and order history. They also include features like login, online payment, prescription upload, and delivery tracking. While these systems are highly functional, they are complex and require backend development and server management.

6.3. Proposed System Analysis

Our system is a simplified version. It does not use a database or backend. All medicine details and cart information are stored directly in the code. Users can browse medicines, add items to a virtual cart, and see an order summary. The system demonstrates core features of an online ordering system without complex infrastructure.

6.4. Functional Analysis

- **User Interaction:** Users can browse a list of medicines, view details, and add them to a virtual cart.
- **Cart Management:** Items in the cart are temporarily stored during the browser session.
- **Order Simulation:** Users can place an order, and the system displays the summary.

6.5. Non-Functional Analysis

- The system is lightweight and can run on any browser without additional software.
- It is educational, showing how online ordering works without a database.
- Security and data storage are limited because there is no backend or database.

6.6. Advantages of the System

- Easy to use and understand.
- Demonstrates the main features of online medicine delivery.
- Can be used as a learning tool for beginners in web development.

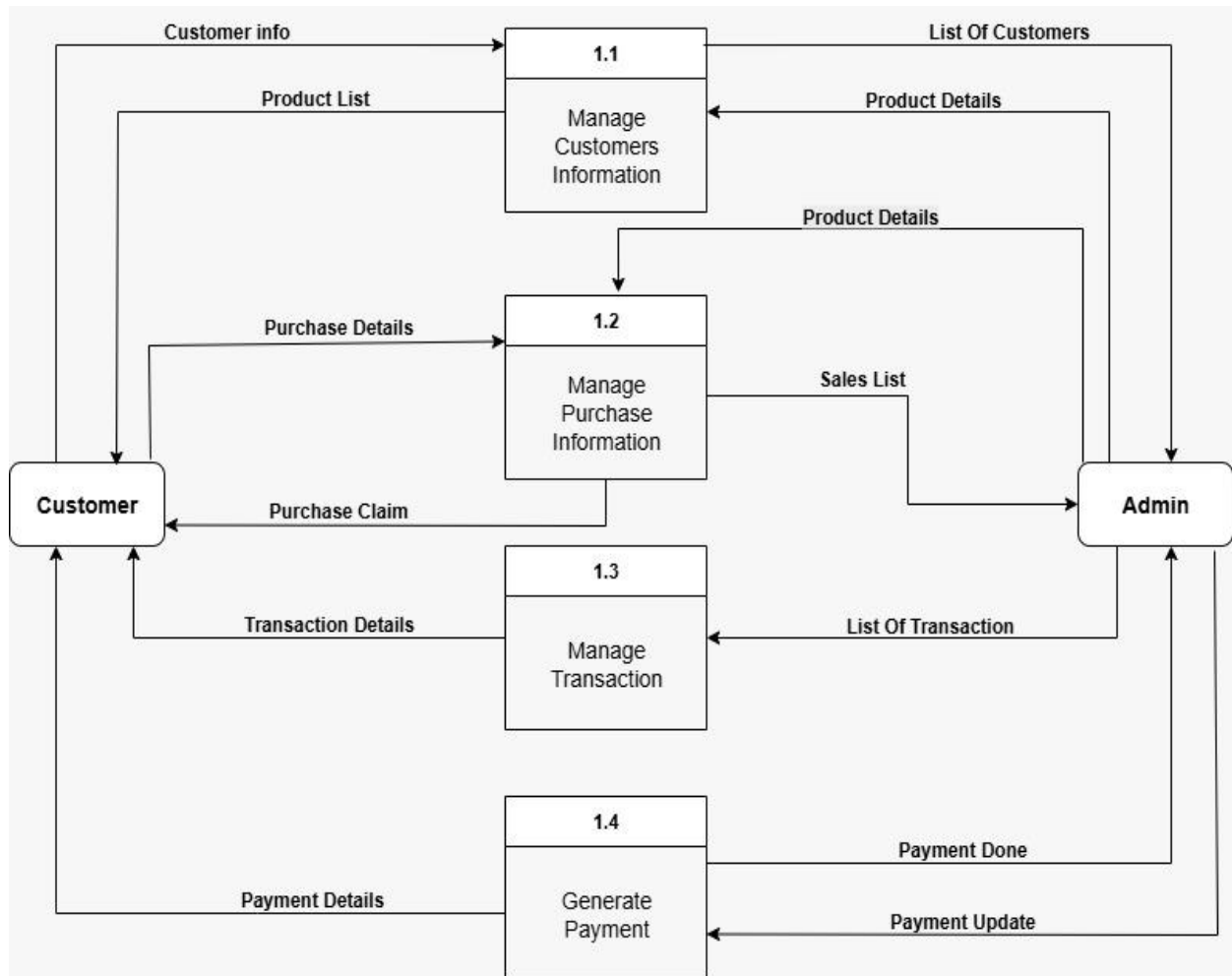
CHAPTER-7

SYSTEM DESIGN

7.1. Data Flow Diagram:

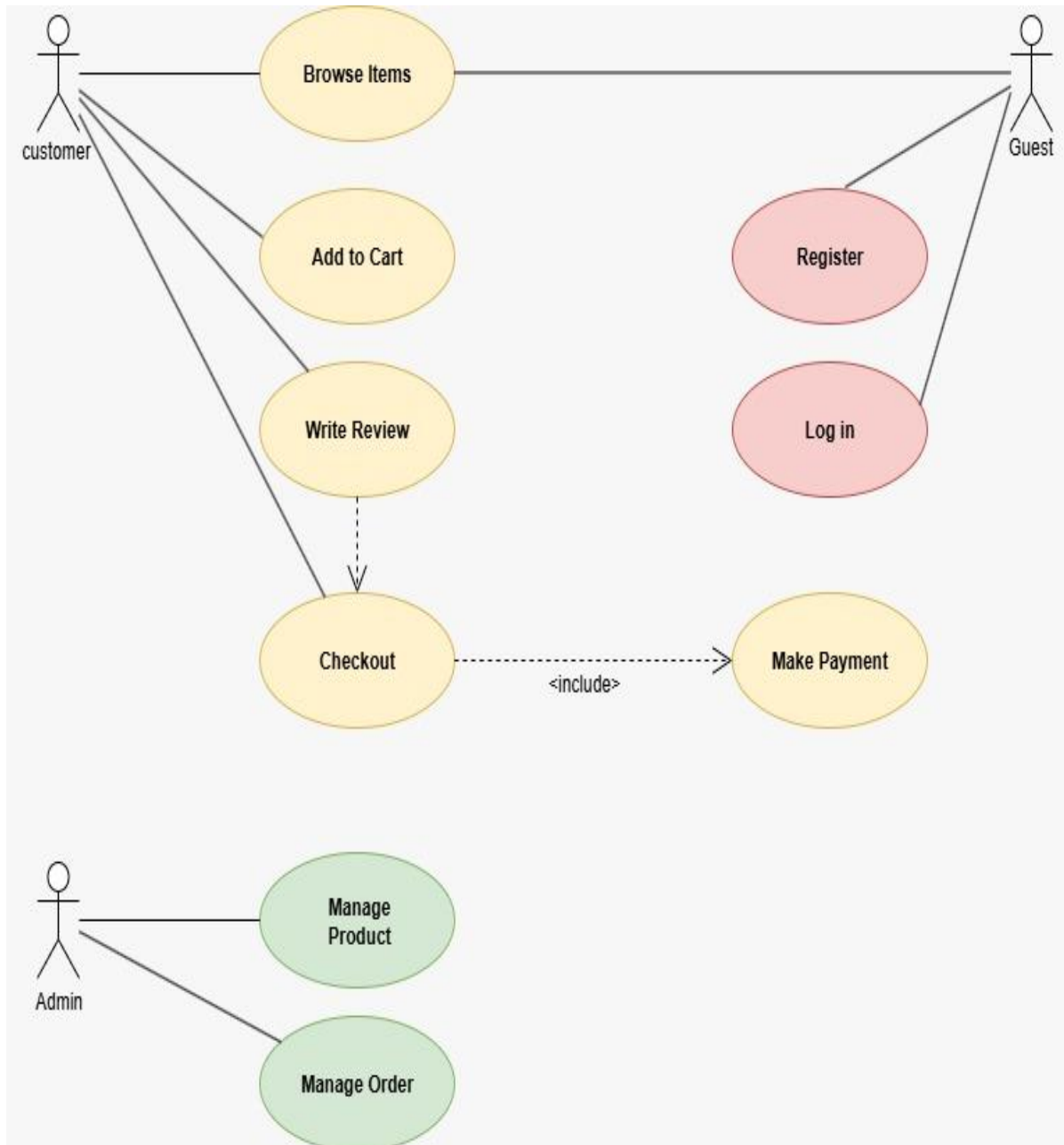
A Data Flow Diagram (DFD) is a visual representation that shows how data moves through a system. It focuses on the flow of information, how it is processed, and where it is stored. DFDs are used in system analysis to understand how a system works and to identify the input, output, and processing steps.

1. **External Entities** (squares/rectangles) → Sources or destinations of data (e.g., Customer, Admin, Pharmacy).
2. **Processes** (circles/ovals) → Transformations applied to data (e.g., *Search Medicine, Place Order*).
3. **Data Stores** (open rectangles) → Where data is held temporarily or permanently (e.g., *Medicine List, Orders*).
4. **Data Flows** (arrows) → Show movement of data between entities, processes, and stores.



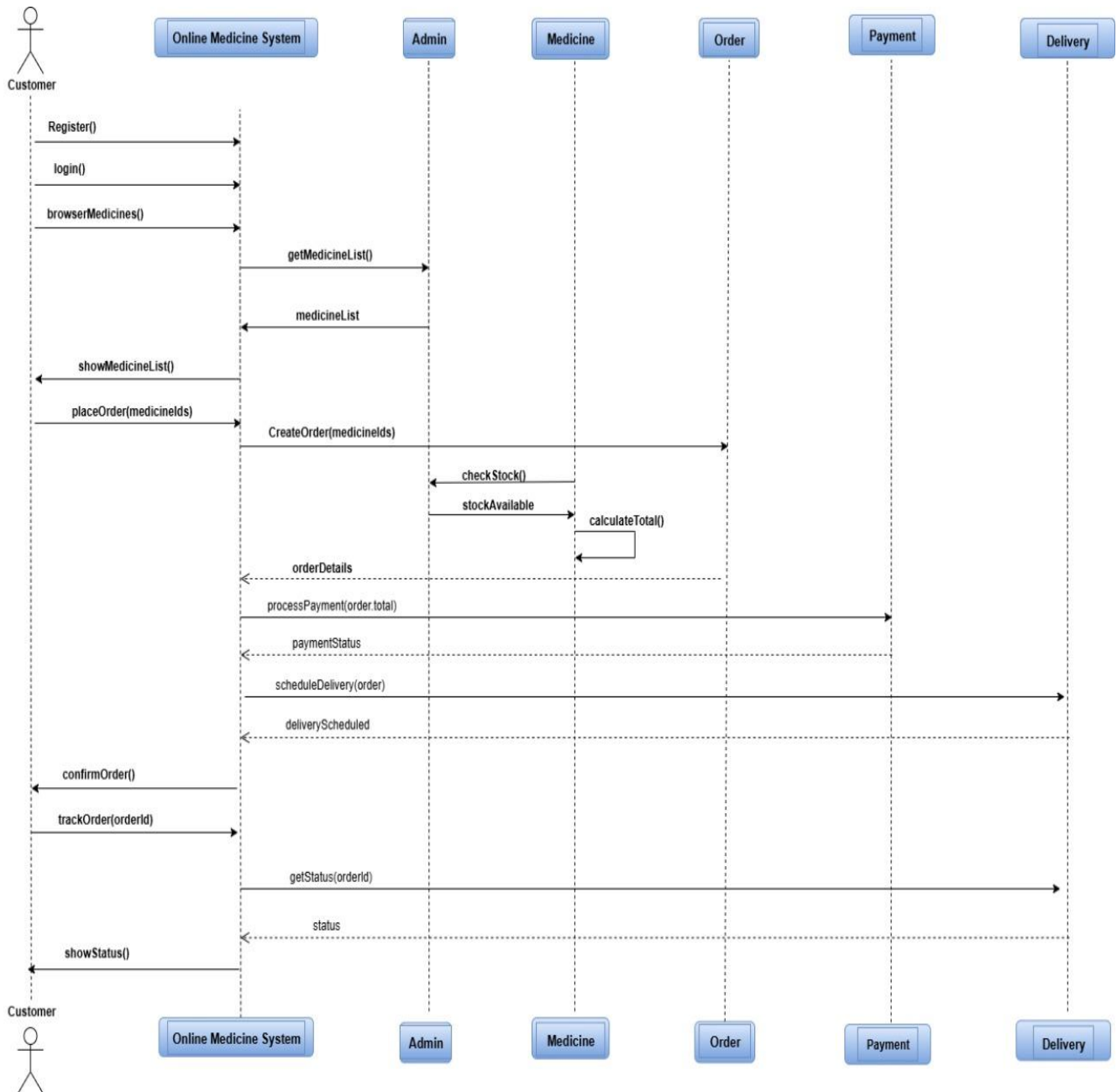
7.2. Use case Diagram

A use case is a way to describe how a user interacts with a system to achieve a specific goal. It shows the steps or actions a user takes and how the system responds. Use cases are often used in system analysis and design to understand the functionality of a system from the user's perspective.



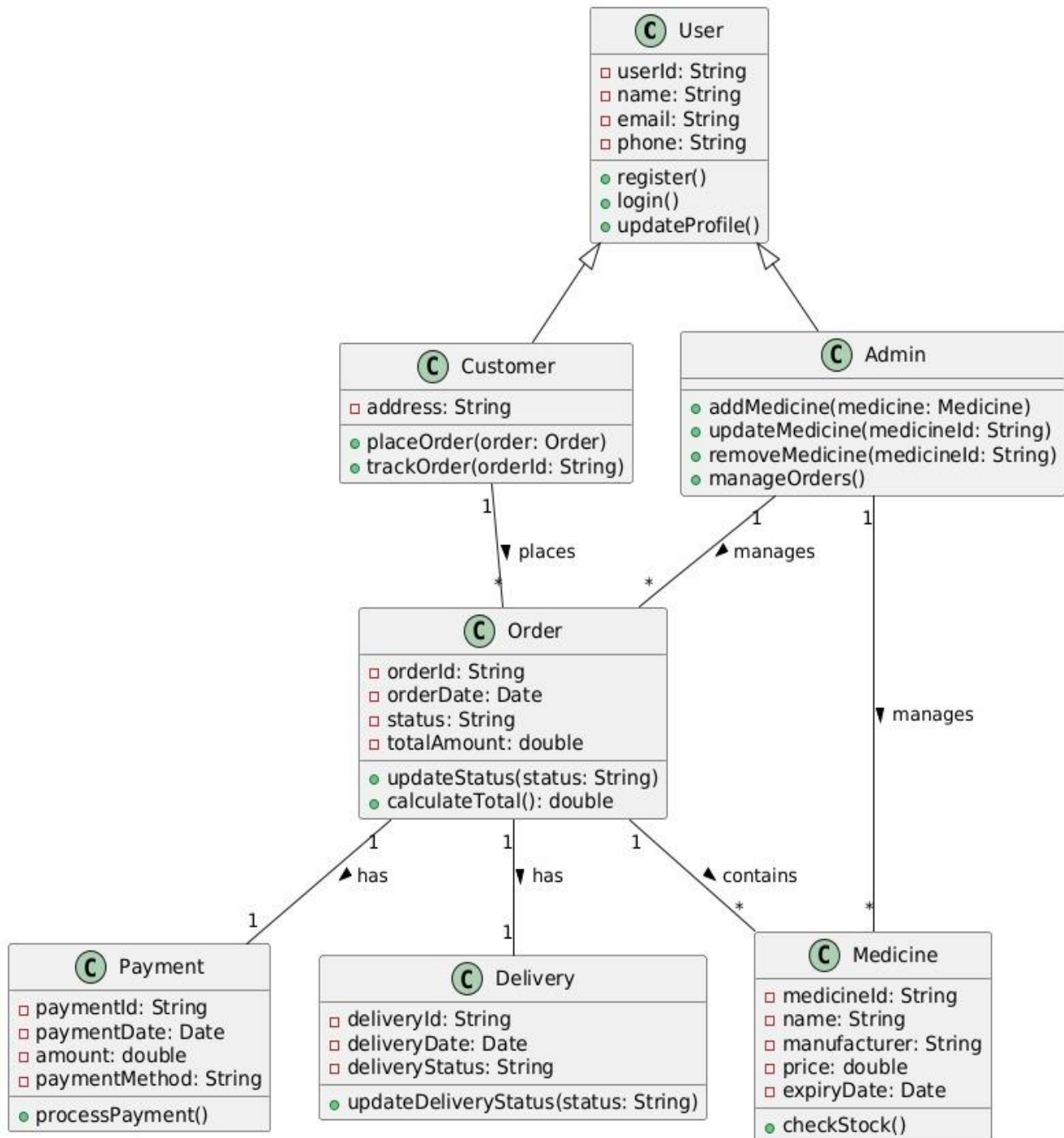
7.3. Sequence Diagram

A Sequence Diagram a step-by-step visual story of how different parts of a system talk to each other to complete a task. A Sequence Diagram is one of the diagrams in UML (Unified Modeling Language) that shows how objects or components interact with each other in a specific order over time. It is mainly used to describe the flow of messages in a system for a particular use case or scenario.



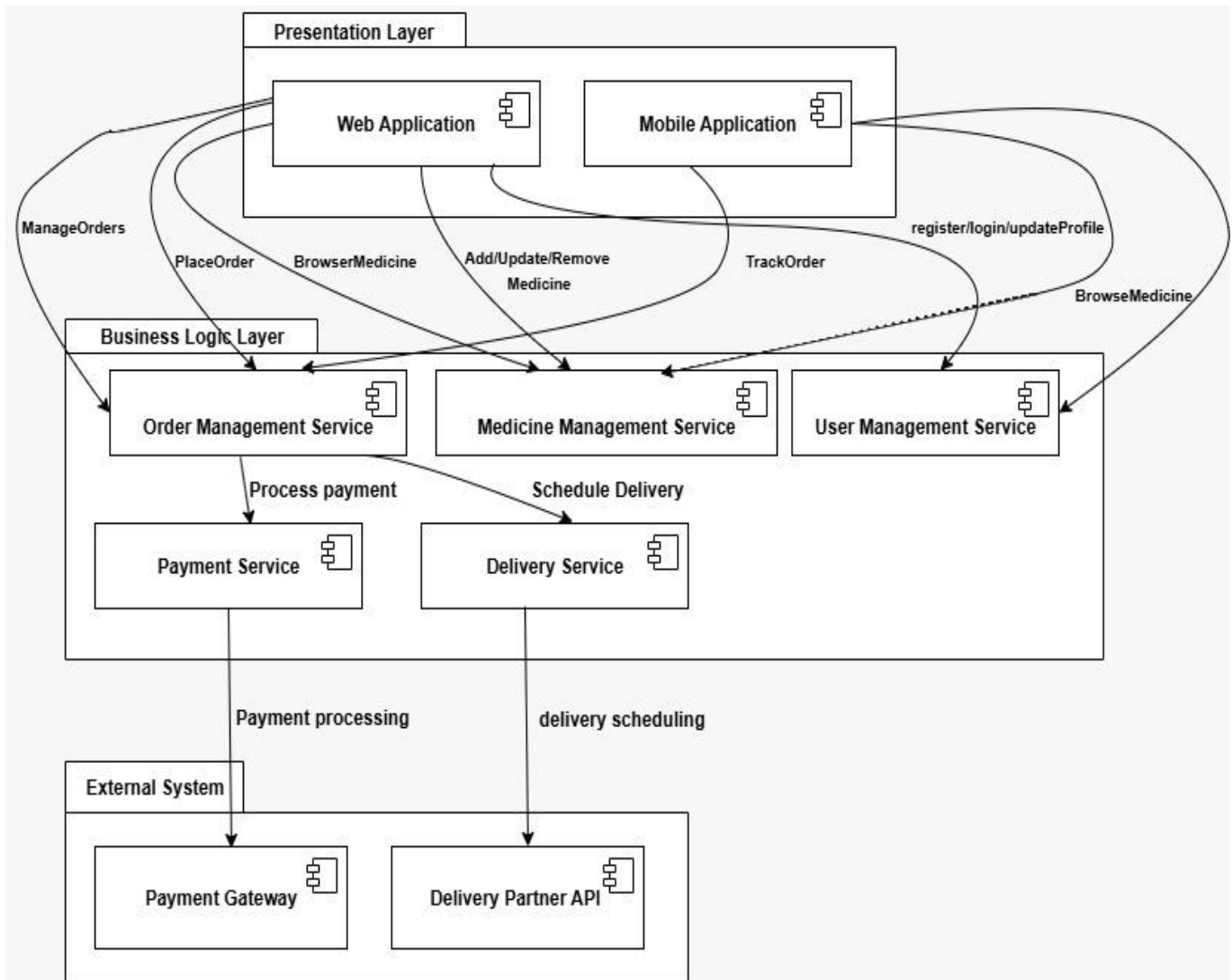
7.4. Class Diagram:

A Class Diagram is a static model that shows the classes in a system, their attributes, methods, and relationships. A Class Diagram is one of the main diagrams in UML (Unified Modeling Language) that shows the static structure of a system.



7.5. Architecture Diagram:

System Architecture is the way a system is structured and organized. It shows how different components of the system interact with each other to perform tasks. For an Online Medicine Delivery System, the architecture defines how users interact with the system, how data is handled, and how orders are processed. Since this project does not use a database, the architecture is simple and mainly front-end based. All data, like medicine details and cart items, are stored temporarily in the code or in the browser's memory.



7.6.UI Design:

The UI design for the Online Medicine Delivery System includes a clean and simple interface built with HTML, CSS, and JavaScript. The homepage welcomes users with a navigation bar and a call-to-action button. The medicine list page displays medicines in a card layout with names, prices, and an "Add to Cart" button. The cart page shows selected items, quantities, and total cost, along with a "Place Order" button. After ordering, a confirmation page thanks the user. The design uses soft medical colors like blue and green, clear fonts, and responsive layouts to ensure a user-friendly experience across all devices.

Create Account

Full Name:

Email:

Password:

Create Account

Login

Email:

Password:

Login

CHAPTER-8

IMPLEMENTATION

Step 1:-

Define system requirements:

The first step in developing a medicine delivery system without using a database is to clearly identify what the system should do. The system must cater to two main types of users, regular users and administrators. For users, the system should provide the ability to view a list of available medicines, search and select specific medicines, place an order for one or more items, and check the status of any orders they have placed. These features ensure that users can interact with the system in a simple and intuitive manner. For administrators, the system must provide functionality to manage the medicine catalog and monitor customer orders. Admins should be able to add new medicines, update stock levels or pricing for existing medicines, view a list of all customer orders, and update the delivery status of those orders. These administrative capabilities are essential for maintaining the accuracy and functionality of the system.

Step 2:-

Choose a data storage method without using a database:

Since the system will not use any database management system, it must rely on simpler forms of data storage. The primary method will be using in-memory data structures during the runtime of the application. These include data types such as lists, dictionaries, arrays, or maps, depending on the programming language used. To preserve data between sessions, optional file-based storage can be used. Flat files such as JSON, CSV, or plain text files can simulate persistent data storage. JSON is particularly useful because it supports complex data structures and is easy to read and write. CSV files are ideal for storing tabular data such as medicine listings. Plain text files may be used for simple logs or notes. These files will serve as a substitute for database tables, allowing data to be saved and loaded without setting up any database server.

Step 3:-

Design data models for the system:

The next step is to structure the data in a way that it can be effectively stored, accessed, and manipulated throughout the system. The medicine model should include attributes such as a unique identifier, the name of the medicine, its price, and the current stock level. This allows the system to track each medicine and perform operations such as displaying, ordering, and updating stock. The order model must include an order identifier, the name of the customer who placed the order, the identifier of the medicine ordered, the quantity requested, and the current status of the order. This status may be pending, shipped, delivered, or cancelled. Optionally, a user model can also be created to store customer details such as name, address, and contact information. This is not mandatory for basic versions of the system, but can be useful for more advanced features.

Step 4:-

Implement core functionalities:

Substep 4.1 – Display Available Medicines.

The system should display all medicines with their details. This includes the medicine's ID, name, price, and current stock. The data is either read from a file or kept in memory while the program runs.

Substep 4.2 – Search Medicines.

Users should be able to search for medicines by typing part of the name or selecting from a category. This feature improves user experience and helps quickly locate desired products.

Substep 4.3-place an order.

Allow users to place an order by selecting a medicine and specifying the quantity. Before accepting the order, the system checks if enough stock is available. If sufficient stock exists, the order is created, stock is reduced, and the order is saved. If not, the system should notify the user that the order cannot be processed.

Substep 4.4-Check order status.

Allow users to check the status of their orders using either their name or a unique order ID. The system should return the current status of the order so users can track its progress.

Substep 4.5-Admin managrment.

Allow administrators to manage the medicine catalog and order statuses. Admins can add new medicines, update existing ones, and remove out-of-stock items. They can also view and update the status of customer orders, changing them from pending to shipped or delivered.

Step 5:-

Manage data saving and loading:

If flat file storage is used, the system should be able to read data from files when the program starts and write updated data back to the files when changes are made. For example, when a new order is placed or stock is updated, the data must be saved so it is not lost after the program ends. JSON files can be used to store lists of medicines and orders in a structured format. Reading from these files at the start of the program ensures that previously stored data is restored, and writing to them after updates ensures data persistence. This simulates basic database behavior without requiring a database engine.

Step 6:-

Build a user interface:

The system interface allows users and administrators to interact with the application. A simple command line interface can be used for basic interactions. Users type menu choices and enter values when prompted. Optionally, a graphical user interface or a basic web interface can be developed using frameworks such as Tkinter for desktop applications or Flask for web applications. Even with these interfaces, no database is required, and all data interactions are handled through in-memory structures and flat files.

Step 7:-**Test the system:**

Once the functionalities are implemented, the system should be thoroughly tested to ensure that it works correctly under various conditions. All features should be tested, including adding and updating medicines, placing orders, viewing medicine details, and checking order statuses. The system should also be tested for boundary conditions such as trying to order more than the available stock, entering invalid input, or updating a non-existent medicine or order. If files are used for data storage, tests should confirm that data is correctly saved and restored. This ensures the system works properly across sessions and does not lose critical information.

Step 8:-**Deploy and use the system:**

After testing is complete, the application can be packaged and deployed. Since there is no database setup required, the system can be run on any computer that supports the programming environment used to build it. The system files, including the program scripts and any flat files, can be copied to any system and executed without the need for database configuration. This makes the application lightweight, portable, and easy to maintain. This approach is ideal for prototypes, learning projects, and small business applications where advanced database features are not necessary.

CHAPTER-9

RESULT AND OUTPUT SCREENSHOTS

9.1.Login Screen:

Create Account

Full Name:

Email:

Password:

Create Account

Login

Email:

Password:

Login

9.2.Main Menu Screen :

 **MEDIPORTAL**(Medicine To Doorstep)

[Home](#) [Contact Us](#) [Order Tracking](#) [W. Cart](#) 0

Welcome, naresh!

Login

Available Medicines



Paracetamol 500 mg / 650 mg

₹30

Add



Ibuprofen 200 mg / 400 mg

₹100

Add



Amoxicillin 250 mg / 500 mg

₹120

Add



Cetirizine 10 mg

₹39

Add



Himalaya Ashvagandha General Wellness 60 Tablets

₹240

Add



Omeprazole – 20 mg or 40 mg

₹60

Add



Aspirin – 75 mg

₹39

Add



Pantoprazole – 40 mg

₹25

Add



Salbutamol Syrup- 2 mg/5 ml

₹150

Add



Domperidone Syrup-5 mg/5 ml

₹120

Add

9.3.Cart Screen

Your Cart

Dettol Lime Fresh Disinfectant Liquid 200 ml ₹149

Pedigree Chicken & Milk Dry Food for Puppy Dog 2.8 kg ₹650

Lifebuoy Total Hand Sanitizer 50 ml ₹50

Total: ₹849



9.4 Order Screen:

Place Your Order

Payment Method:

Cash on Delivery ▼

Full Name:

naresh

Phone Number:

8666885643

Pincode:

534201

Delivery Address:

Bhimavaram,DNR college Attached Boy's Hostel

Place Order

9.5.Order Tracking Screen:

Track Your Order

Enter your Order ID:

ORD963650

Track

Order Status: Order Placed

CHAPTER-10

CHALLENGES FACED

1. Data Storage Limitations

- Without a database, all medicine, user, and order information has to be stored in temporary structures like arrays, JSON files, or local storage.
- This limits the amount of data that can be stored and makes it difficult to maintain long-term records.
- Data persistence across sessions can be unreliable, especially if using only client-side storage.

2. Scalability Issues

- Handling a growing number of users and orders becomes very difficult.
- Adding more medicines or users requires manually updating the code or files, which is inefficient.

3. Data Consistency & Integrity

- Without a database, ensuring data accuracy is challenging.
- Duplicate entries, incorrect stock levels, or user information inconsistencies may occur easily.

4. Security Concerns

- Sensitive information like user details, addresses, and order history may be stored insecurely in local files or client-side storage.
- Lack of proper authentication and authorization mechanisms increases the risk of data breaches.

5. Real-time Updates

- Updating medicine availability, order status, or prices in real-time is almost impossible without a centralized database.
- Users may see outdated information if multiple users access the system simultaneously.

6. Limited Functionality

- Features like search, filtering, and sorting medicines become harder to implement efficiently.
- Generating reports (sales, stock levels) is cumbersome without a database query system.

7. Error Handling

- Managing errors like invalid orders or stock shortages requires complex code logic, as there is no database to help with validations.

8. Maintenance Difficulties

- Adding, removing, or updating medicines, users, and orders requires direct code changes or file modifications.
- This is time-consuming and prone to human errors.

9. Multi-user Management

- Supporting multiple users simultaneously is very difficult.
- Without a database, keeping track of each user's orders or cart becomes inefficient.

10. No Transaction Management

Implementing secure payment and order confirmation is challenging.

11. Data Loss Risk

System crashes or file corruption can result in permanent loss of data.

12. Reporting and Analytics Limitations

Generating sales, stock, or user activity reports is cumbersome.

CHAPTER-11

FUTURE SCOPE / MODIFICATIONS

1. **Integration with Local Pharmacy APIs:**
 - Instead of using a database, integrate with local pharmacy APIs to fetch real-time medicine availability and prices.
2. **Offline Mode with Data Caching:**
 - Implement caching on the client side (e.g., in the browser's local storage) to store medicine details temporarily for faster access and offline usage.
3. **Use of File-Based Storage:**
 - Use lightweight file formats like JSON or XML stored locally or on the server to manage medicine data without a formal database.
4. **Improved User Interface:**
 - Enhance UI/UX for easier medicine search, filtering by symptoms, brands, or categories, and clearer order tracking.
5. **Automated Reminders & Alerts:**
 - Implement features to remind users to reorder medicines based on past purchases or typical medication cycles.
6. **Integration with Payment Gateways:**
 - Add multiple payment options (UPI, credit/debit cards, wallets) without needing a database for order processing.
7. **Real-time Order Status:**
 - Use simple server-side scripts or APIs to update order status (accepted, dispatched, delivered) dynamically.
8. **Voice Search & Assistance:**
 - Implement voice commands for searching medicines and placing orders, enhancing accessibility.
9. **Security Enhancements:**
 - Use encryption and secure communication protocols (HTTPS) to protect user data during transactions, even if no database is involved.
10. **Feedback & Rating System:**
 - Allow users to submit feedback/rating stored in local files or sent via email to the admin.
11. **SMS/Email Notifications:**
 - Send order confirmation and delivery updates via SMS or email using third-party services without needing a database.

CHAPTER-12

CONCLUSION

The Online Medicine Delivery System developed using basic web technologies HTML, CSS, and JavaScript without the use of a database showcases a simple yet functional approach to simulating the process of ordering medicines online. It provides users with the ability to browse a catalog of medicines, view relevant information such as names, descriptions, and prices, add items to a virtual cart, and simulate placing an order. All of these features are implemented entirely on the client side, making the system lightweight, easy to deploy, and accessible through any standard web browser without requiring server-side components. However, the absence of a backend database and server infrastructure introduces several notable limitations. The system cannot store user data or order history persistently, as all information is held in temporary JavaScript variables or browser storage mechanisms like `localStorage`. As a result, data is lost when the browser session ends or the page is refreshed. This restricts the system's ability to support user authentication, multi-user functionality, session tracking, and personalized experiences, all of which are essential for a fully functional e-commerce platform. In addition, the lack of a database makes it difficult to implement real-time inventory management, where medicine stock levels can be updated dynamically based on user activity. Without backend processing, scalability is limited, and the system is not equipped to handle multiple users or large volumes of data simultaneously. Furthermore, without proper backend validation and security mechanisms, the system is vulnerable to issues related to data integrity, unauthorized access, and security breaches, making it unsuitable for handling sensitive information such as prescriptions or payment data. Despite these challenges, the system serves as a valuable educational tool and a strong foundation for learning. It is particularly useful for students, beginners in web development, or those creating prototype applications. The project illustrates fundamental concepts in front-end development, including user interface design, dynamic data handling, and user interaction workflows, all in the context of a healthcare-related application. It effectively bridges theoretical knowledge with hands-on practice, making abstract concepts more tangible. For full-scale deployment or real-world usage, it would be necessary to extend the system by integrating a relational or NoSQL database to manage persistent data, adding secure login systems, incorporating encryption techniques, and enabling payment gateway integration to support secure transactions. Additionally, features such as admin panels, order tracking, inventory dashboards, and reporting tools would need to be developed to enhance usability and manage operations effectively. In conclusion, while the current version of the Online Medicine Delivery System is limited in scope, it successfully demonstrates the core principles of how an e-commerce platform functions. It lays a solid groundwork for further development and serves as a stepping stone toward building more robust, secure, and scalable systems in the future. With proper enhancements, the prototype can evolve into a fully-featured application capable of delivering real value in the healthcare domain.

CHAPTER-13

REFERENCES

Research Papers

1. **"Towards a Secured Smart IoT Using Lightweight Blockchain: An Aim to Secure Pharmacy Products"**
This paper explores integrating IoT with lightweight blockchain to secure pharmaceutical product delivery, offering insights into decentralized systems that minimize database usage.
2. **"The Impact of Pharmacybernetic in Reducing Medication Error"**
Discusses the implementation of Electronic Drug Information Records (EDIR) and the concept of 'Pharmacybernetic' to reduce medication errors, which can be relevant for systems aiming to operate with minimal databases.
3. **"A Secure Electronic Prescription System Using Steganography with Encryption Key Implementation"**
This paper presents a secure e-prescription system that embeds prescription data within images using steganography, combined with encryption keys to enhance confidentiality. It emphasizes secure transmission without relying on large centralized databases, aligning with systems designed for lightweight and decentralized healthcare data exchange.

GitHub Repositories

1. **GitHub - amancsc8/Online-Medicine-Delivery-System-Vatiwala**
 - A comprehensive Angular and Spring Boot-based online medicine delivery system. While it uses Oracle as a database, the project's structure and codebase can provide valuable insights for building systems with minimal database dependencies.
2. **GitHub - Varshini-E/Pharmacy-Management-System**
 - A web-based application implemented using PHP and MySQL. Studying its architecture can offer perspectives on integrating minimal database solutions in pharmacy management systems.
3. **GitHub - nikhil-188/OnlineDrugStore**
 - A web application that allows users to order medicines online. While it employs a database, the project's approach to inventory management and user authentication can be adapted for systems with reduced database reliance.
 - [mediportal2-design/mediportal](#)

APPENDICES-I

SOURCE CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-
scale=1" />
<title>Pharmacy Delivery System</title>
<style>
  body {
    font-family: Arial, sans-serif;
    margin:0; padding:0;
    background:#FFFFFF;
  }
  header {
    background: #014D4E;
    color: white;
    padding: 0.8rem 2rem;
  }
  .nav-bar {
    display: flex;
    align-items: center;
    justify-content: space-between;
    max-width: 900px;
    margin: 0 auto;
  }
  .logo {
    font-weight: bold;
    font-size: 1.3rem;
  }
  .logo {
    display: flex;
    align-items:center ;
    font-weight: bold;

    font-size: 1.3rem;
    gap: 0.5rem;
```



```

}
.logo-img {
  height: 90px;
  width: 90px;
  object-fit: contain;
}
.nav-links {
  list-style: none;
  display: flex;
  gap: 1.5rem;
  margin: 0;
  padding: 0;
}
.nav-links li a {
  color: white;
  text-decoration: none;
  font-weight: 600;
  font-size: 1rem;
  transition: color 0.3s ease;
  cursor: pointer;
}
.nav-links li a:hover {
  color: #cce5ff;
}
.cart-link {
  position: relative;
}
#cartCount {
  background: #28a745;
  border-radius: 50%;
  padding: 0 7px;
margin-left: 5px;
  font-size: 0.85rem;
  font-weight: bold;
  vertical-align: top;
}
main {
  max-width: 900px;
  margin: 2rem auto;
  padding: 0 1rem;

```

```

}
h1,h2 {
  margin:0 0 1rem 0;
}
form {
  max-width:400px;
  margin: 1rem auto;
  background:#fff;
  padding:1rem 2rem;
  border-radius:8px;
  box-shadow: 0 2px 6px rgb(0 0 0 / 0.1);
}
label {
  display:block;
  margin-top:1rem;
}
input, textarea {
  width: 100%;
  padding:0.5rem;
  margin-top:0.3rem;
  border-radius:4px;
  border:1px solid #ccc;
  font-size:1rem;
}
button {
  margin-top:1rem;
  background:#014D4E;
  color:white;
  border:none;
  padding:0.5rem 1.5rem;
  font-size:1rem;
  border-radius:0.5px;
  cursor:pointer;
}
button:disabled {
  background:#7ea9e1;
  cursor: not-allowed;
}
.error {
  color:red;

```

```

    margin-top:0.5rem;
}
.medicine-list {
    display: grid;
    grid-template-columns: repeat(auto-fill,minmax(160px,1fr));
    gap: 1rem;
    margin-top: 2rem;
}
.medicine-card {
    background:#FFFFFF;
    border-radius:8px;
    padding:1rem;
    box-shadow: 0 1px 4px rgb(0 0 0 / 0.1);
    text-align:center;
    display:flex;
    flex-direction: column;
    justify-content: space-between;
}
.medicine-card img {
    max-width: 100px;
    height: 100px;
    object-fit: contain;
    margin: 0 auto 0.5rem;
}
.price {
    font-weight:bold ;
    color: #000000;
    margin: 0.5rem 0;
}
.cart {
    margin-top: 2rem;
    background:#fff;
    border-radius:8px;
    padding:1rem 2rem;
    box-shadow: 0 1px 5px rgb(0 0 0 / 0.1);
}
.cart-item {
    display:flex;
    justify-content: space-between;
    align-items: center;

```

```

        margin-bottom: 0.5rem;
    }
    .cart-item span {
        flex:1;
    }
    .quantity-controls button {
        margin: 0 0.2rem;
        padding: 0.2rem 0.5rem;
        font-weight:bold;
    }
    #orderSection {
        margin-top: 2rem;
        background:#fff;
        border-radius:8px;
        padding:1rem 2rem;
        box-shadow: 0 1px 5px rgb(0 0 0 / 0.1);
    }
    #qrCodeContainer {
        text-align: center;
        margin-top: 1rem;
    }
    #logoutBtn {
        float: right;
        margin-bottom: 1rem;
        background: #ff0000;
    }
    #logoutBtn:hover {
background: #c82333;
    }
    /* Registration/Login container */
    #authContainer {
        max-width: 420px;
        margin: 2rem auto;
        background: white;
        border-radius: 8px;
        padding: 1rem 2rem;
        box-shadow: 0 2px 6px rgb(0 0 0 / 0.1);
    }
    #authContainer h2 {
        text-align: center;

```

```

    margin-bottom: 1rem;
}
#registerForm, #loginFormAuth {
    margin-bottom: 2rem;
}
#registerForm button, #loginFormAuth button {
    width: 100%;
}
#authContainer input {
    margin-top: 0.5rem;
}
/* Modal styles */
#orderTrackingModal {
    display: none; /* hidden by default */
    position: fixed;
    z-index: 999;
    left: 0; top: 0;
    width: 100%; height: 100%;
    overflow: auto;
    background-color: rgba(0,0,0,0.5);
}
#orderTrackingModalContent {
    background: white;
    margin: 10% auto;
    padding: 1.5rem 2rem;
    border-radius: 8px;
    max-width: 400px;
    position: relative;
    box-shadow: 0 2px 8px rgba(0,0,0,0.3);
}
#orderTrackingModalContent h3 {
    margin-top: 0;
}
#closeModalBtn {
    position: absolute;
    top: 10px; right: 15px;
    font-size: 1.5rem;
    font-weight: bold;
    color: #333;
    cursor: pointer;
}

```

```

    }
    #trackStatus {
        margin-top: 1rem;
        font-weight: bold;
        color: #007bff;
    }
</style>
</head>
<body>
<header>
<div class="logo">
    
    <div class="logo">MEDIPORTAL (Medicine To Doorstep)</div>
    <nav class="nav-bar">
<ul class="nav-links">
    <li><a href="#appSection" id="homeLink">Home</a></li>
    <li><a href="#contactSection" id="contactLink">Contact
Us</a></li>
    <li><a id="orderTrackingLink">Order Tracking</a></li>
    <li class="cart-link">
        <a href="#cartSection">
            Cart <span id="cartCount">0</span>
        </a>
    </li>
</ul>
</nav>
</header>
<main>
    <!-- AUTH CONTAINER: Registration + Login -->
    <section id="authContainer">
<h2>Create Account</h2>
        <form id="registerForm">
            <label>Full Name:
                <input type="text" id="fullname" placeholder="Full Name"
required />
            </label>
            <label>Email:
                <input type="email" id="emailReg" placeholder="Email"
required />
            </label>

```

```

        <label>Password:
            <input type="password" id="passwordReg"
placeholder="Password" required />
        </label>
        <button type="submit">Create Account</button>
    </form>
    <h2>Login</h2>
    <form id="loginFormAuth">
        <label>Email:
            <input type="email" id="emailLogin" placeholder="Email"
required />
        </label>
        <label>Password:
            <input type="password" id="passwordLogin"
placeholder="Password"
required />
        </label>
        <button type="submit">Login</button>
    </form>
</section>
<!-- Main app area (hidden initially) -->
<section id="appSection" style="display:none;">
    <button id="logoutBtn">Logout</button>
    <h1>Welcome, <span id="userNameDisplay"></span>!</h1>
    <h2>Available Medicines</h2>
<div style="margin: 1rem 0;">
    <input type="text" id="searchInput" placeholder="Search
medicines..." style="width: 100%; padding: 0.5rem; font-size: 1rem;
border: 1px solid #ccc; border-radius: 4px;" />
</div>
    <div class="medicine-list" id="medicineList">
        <!-- Medicines dynamically inserted here -->
    </div>
    <section class="cart" id="cartSection">
        <h2>Your Cart</h2>
        <div id="cartItemsContainer">
            <!-- Cart items listed here -->
        </div>
        <div id="cartTotal"></div>
    </section>
    <section id="orderSection">

```

```

        <h2>Place Your Order</h2>
        <form id="orderForm">
<b><label>Payment Method:</b>
    <b> <select id="paymentMethod" required></b>
        <option value="">-- Select Payment Method --</option>
        <option value="cod">Cash on Delivery</option>
        <!-- Future: <option value="online">Online Payment</option> -->
    </select>
</label>
        <label>Full Name:
            <input type="text" id="deliveryFullName"
placeholder="Enter your full name" required />
        </label>
        <label>Phone Number:
            <input type="tel" id="deliveryPhone" placeholder="Enter
your phone number" pattern="[0-9]{10}" title="Enter a 10-digit
phone number" required />
        </label>
        <label>Pincode:
            <input type="text" id="deliveryPincode"
placeholder="Enter your pincode" pattern="[0-9]{5,6}" title="Enter
a valid pincode" required />
        </label>
        <label>Delivery Address:
            <textarea id="deliveryAddress" rows="3"
placeholder="Enter your delivery address" required></textarea>
        </label>
            <button type="submit" id="placeOrderBtn" disabled>Place
Order</button>
        </form>
        <div id="orderError" class="error"></div>
    </section>
<section id="qrCodeContainer"></section>
</section>
    <!-- Contact Section -->
    <section id="contactSection" style="max-width:900px; margin:
2rem auto; padding: 0 1rem;">
        <h2>Contact Us</h2>
        <p>For any inquiries or support, please email us at <a
href="mailto:support@Mediportal.com">Mediportal2@gmail.com</a> or
98466xxxx7.</p>
    </section>
</main>

```



```

<!-- Order Tracking Modal -->
<div id="orderTrackingModal">
  <div id="orderTrackingModalContent">
    <span id="closeModalBtn">&times;</span>
    <h3>Track Your Order</h3>
    <form id="trackingForm">
      <label>Enter your Order ID:
        <input type="text" id="trackOrderId" placeholder="Order
ID" required />
      </label>
      <button type="submit">Track</button>
    </form>
    <div id="trackStatus"></div>
  </div>
</div>
<script
src="https://cdn.jsdelivr.net/npm/qrcode/build/qrcode.min.js"></sc
ript>
<script>
  const medicineListEl = document.getElementById('medicineList');
  const cartItemsContainer =
document.getElementById('cartItemsContainer');
  const cartCountEl = document.getElementById('cartCount');
  const cartTotalEl = document.getElementById('cartTotal');
  const placeOrderBtn = document.getElementById('placeOrderBtn');
  const orderErrorEl = document.getElementById('orderError');
  const qrCodeContainer =
document.getElementById('qrCodeContainer');
  const deliveryFullNameEl =
document.getElementById('deliveryFullName');
  const deliveryPhoneEl = document.getElementById('deliveryPhone');
  const deliveryPincodeEl =
document.getElementById('deliveryPincode');
  const deliveryAddressEl =
document.getElementById('deliveryAddress');
  const orderForm = document.getElementById('orderForm');
  const searchInput = document.getElementById('searchInput');
  searchInput.addEventListener('input', () => {
    const searchTerm = searchInput.value.trim();
    renderMedicines(searchTerm);
  });
  // Authentication elements

```

```

const authContainer = document.getElementById('authContainer');
const appSection = document.getElementById('appSection');
const registerForm = document.getElementById('registerForm');
const loginFormAuth = document.getElementById('loginFormAuth');
const utBtn = document.getElementById('logoutBtn');
const userNameDisplay =
document.getElementById('userNameDisplay');
    // Order Tracking Modal Elements
const orderTrackingModal =
document.getElementById('orderTrackingModal');
const orderTrackingLink =
document.getElementById('orderTrackingLink');
const closeModalBtn = document.getElementById('closeModalBtn');
const trackingForm = document.getElementById('trackingForm');
const trackOrderIdInput =
document.getElementById('trackOrderId');
const trackStatusEl = document.getElementById('trackStatus');
    // Sample medicines data
const medicines = [
{
    id: 'med001',
    name: 'Paracetamol          500 mg / 650 mg',
    price: 30,
    image: 'me.png'
},
{
    id: 'med002',
    name: 'Ibuprofen 200 mg / 400 mg',
    price: 100,
    image: 'par.png'
},
{
    id: 'med003',
    name: 'Amoxicillin 250 mg / 500 mg',
    price: 120,
    image: 'am.png'
},
{
    id: 'med004',
    name: 'Cetirizine          10 mg',

```

```

        price: 39,
        image: 'v.png'
    },
    {
        id: 'med005',
        name: 'Himalaya Ashvagandha General Wellness, 60 Tablets',
        price: 240,
        image: 'his.png'
    },
    {
        id: 'med006',
        name: 'Omeprazole - 20 mg or 40 mg ',
        price: 50,
        image: 'om.png'
    },
    {
        id: 'med007',
        name: 'Aspirin - 75 mg',
        price: 39,
        image: 'as.png'
    },
    {
        id: 'med008',
        name: 'Pantoprazole - 40 mg',
        price: 25,
        image: 'pa.png'
    },
    {
        id: 'med009',
        name: 'Salbutamol Syrup -2 mg/5 ml',
        price: 150,
        image: 'sal.png'
    },
    {
        id: 'med010',
        name: 'Domperidone Syrup-5 mg/5 ml',
        price: 120,
        image: 'dom.png'
    },
    {

```

```

    id: 'med011',
      name: 'Levocetirizine Syrup-2.5 mg/5 ml  ',
price: 95,
      image: 'lev.png'
},
{
  id: 'med012',
    name: 'Cefixime Dry Syrup-100 mg/5 ml ',
    price: 175,
    image: 'cef.png'
},
{
  id: 'med0013',
    name: 'Condom',
    price: 130,
    image: 'co.png'
},
{
  id: 'med014',
    name: 'First Aid Kit',
    price: 300,
    image: 'fak.png'
},
{
  id: 'med015',
    name: 'Vicks Vaporub Xtra Strong 50 ml',
    price: 200,
    image: 'vic.png'
},
{
  id: 'med016',
    name: 'Dettol Lime Fresh Disinfectant Liquid 200 ml',
    price: 149,
    image: 'det.png'
},
{
  id: 'med017',
    name: 'Pedigree Chicken & Milk Dry Food for Puppy Dog 2.8
kg',
    price: 650,

```

```

        image: 'pg.png'
    },
    {
        id: 'med018',
        name: 'Lifebuoy Total Hand Sanitizer 50 ml',
        price: 50,
        image: 'lb.png'
    },
    {
        id: 'med019',
        name: 'Digital Thermometer',
        price: 100,
        image: 'dt.png'
    },
    {
        id: 'med020',
        name: 'CERELAC Baby Food Cereal | Multigrain & Fruits for 12
            to 24 Months Baby | Source of Iron',
        price: 330,
        image: 'cer.png'
    },
    {
        id: 'med021',
        name: 'Prega News Pregnancy Test Kit',
        price: 60,
        image: 'prg.png'
    }
];

let cart = {};
let orders = {}; // Store orders by orderId
let currentUser = null;
// Utility functions for localStorage persistence
function saveCart() {
    if(currentUser) {
localStorage.setItem(cart_`${currentUser.email}`,
JSON.stringify(cart));
    }
}

function loadCart() {
    if(currentUser) {

```

```

        const storedCart =
localStorage.getItem(cart_`${currentUser.email}`);
        if(storedCart) {
            cart = JSON.parse(storedCart);
        } else {
            cart = {};
        }
    } else {
        cart = {};
    }
}
function saveOrders() {
    localStorage.setItem('orders', JSON.stringify(orders));
}
function loadOrders() {
    const storedOrders = localStorage.getItem('orders');
    if(storedOrders) {
        orders = JSON.parse(storedOrders);
    }
}
// Generate unique order ID
function generateOrderId() {
    return 'ORD' + Math.floor(Math.random() *
1000000).toString().padStart(6, '0');
}
// Render medicine list
// Render medicine list with optional search filter
function renderMedicines(searchTerm = '') {
    medicineListEl.innerHTML = '';
    const filteredMedicines = medicines.filter((med =>
        med.name.toLowerCase().includes(searchTerm.toLowerCase()))
);
    if (filteredMedicines.length === 0) {
        medicineListEl.innerHTML = '<p>No medicines found.</p>';
        return;
    }
    filteredMedicines.forEach((med => {
        const card = document.createElement('div');
        card.className = 'medicine-card';
        const quantity = cart[med.id] || 0;
        card.innerHTML = `

```

```

        
        <h4>${med.name}</h4>
        <div class="price">₹${med.price}</div>
        <div id="controls_${med.id}">
            ${quantity} === 0
            ? <button data-id="${med.id}" class="add-
btn">Add</button>
            `;
        medicineListEl.appendChild(card);
    });
}

// Update cart count and render cart items
function updateCart() {
    let totalItems = 0;
    let totalPrice = 0;
    cartItemsContainer.innerHTML = '';
    for (const medId in cart) {
        const quantity = cart[medId];
        const med = medicines.find(m => m.id === medId);
        if (!med) continue;
        totalItems += quantity;
        totalPrice += med.price * quantity;
        const itemDiv = document.createElement('div');
        itemDiv.className = 'cart-item';
        itemDiv.innerHTML = `
            <span>${med.name}</span>
            <span>₹${med.price}</span>
            <div class="quantity-controls">
                <button data-action="decrease" data-id="${med.id}">-
</button>
                <span>${quantity}</span>
                <button data-action="increase" data-
id="${med.id}">+</button>
                <button data-action="remove" data-
id="${med.id}"> </button>
            </div>
            `;
        cartItemsContainer.appendChild(itemDiv);
    }
}

```

```

        cartCountEl.textContent = totalItems;
        cartTotalEl.textContent = Total: ₹${totalPrice};
        // Enable/disable place order button based on cart and form
validation
        checkOrderButtonState();
        saveCart();
    }
    // Add medicine to cart
    medicineListEl.addEventListener('click', e => {
        if(e.target.tagName === 'BUTTON') {
            const medId = e.target.getAttribute('data-id');
            if (cart[medId]) {
                cart[medId]++;
            } else {
                cart[medId] = 1;
            }
            updateCart();
        }
    });
    // Cart quantity controls
    cartItemsContainer.addEventListener('click', e => {
        const medId = e.target.getAttribute('data-id');
        const action = e.target.getAttribute('data-action');
        if (!medId || !action) return;
        if (action === 'increase') {
            cart[medId]++;
        } else if (action === 'decrease') {
            if(cart[medId] > 1) {
                cart[medId]--;
            } else {
                delete cart[medId];
            }
        } else if (action === 'remove') {
            delete cart[medId];
        }
        updateCart();
    });
    // Check if order form inputs are valid and cart is not empty to
enable Place Order button
    function checkOrderButtonState() {
        const fullNameValid = deliveryFullNameEl.value.trim().length >0;

```



```

    const phoneValid = deliveryPhoneEl.validity.valid;
    const pincodeValid = deliveryPincodeEl.validity.valid;
    const addressValid = deliveryAddressEl.value.trim().length > 0;
    const cartNotEmpty = Object.keys(cart).length > 0;
    placeOrderBtn.disabled = !(fullNameValid && phoneValid &&
pincodeValid && addressValid && cartNotEmpty);
  }
  // Listen for input changes in delivery form to update button
state
  [deliveryFullNameEl, deliveryPhoneEl, deliveryPincodeEl,
deliveryAddressEl].forEach(el => {
    el.addEventListener('input', checkOrderButtonState);
  });
  // Handle order form submission
  orderForm.addEventListener('submit', e => {

    e.preventDefault();
    orderErrorEl.textContent = '';
    // Validate again before processing
    if(placeOrderBtn.disabled) {
      orderErrorEl.textContent = 'Please fill all delivery details
correctly and add items to your cart.';
      return;
    }
    const orderId = generateOrderId();
    const orderDetails = {
      id: orderId,
      userEmail: currentUser.email,
      items: {...cart},
      delivery: {
        fullName: deliveryFullNameEl.value.trim(),
        phone: deliveryPhoneEl.value.trim(),
        pincode: deliveryPincodeEl.value.trim(),
        address: deliveryAddressEl.value.trim()
      },
      status: 'Order Placed',
      date: new Date().toISOString()
    };
    orders[orderId] = orderDetails;
    saveOrders();
    // Clear cart and form

```

```

    cart = {};
    updateCart();
    orderForm.reset();
    checkOrderButtonState();
    // Show QR code with order ID
    qrCodeContainer.innerHTML = <h3>Your Order ID:
    ${orderId}</h3><div id="qrcode"></div>;
    QRCode.toCanvas(document.getElementById('qrcode'), orderId,
    function (error) {
        if (error) console.error(error);
    });
    alert(Order placed successfully! Your Order ID is ${orderId}.
    Use it to track your order.);
    });
    // Order Tracking Modal handlers
    orderTrackingLink.addEventListener('click', () => {
        orderTrackingModal.style.display = 'block';
        trackOrderIdInput.value = '';
        trackStatusEl.textContent = '';
    });
    closeModalBtn.addEventListener('click', () => {
        orderTrackingModal.style.display = 'none';
    });
    window.addEventListener('click', (e) => {
        if(e.target === orderTrackingModal) {
            orderTrackingModal.style.display = 'none';
        }
    });
    trackingForm.addEventListener('submit', e => {
        e.preventDefault();
        const orderId = trackOrderIdInput.value.trim();
        if(!orderId) {
            trackStatusEl.textContent = 'Please enter a valid Order ID.';
            return;
        }
        if(orders[orderId]) {
            const order = orders[orderId];
            trackStatusEl.textContent = Order Status: ${order.status};
        } else {
            trackStatusEl.textContent = 'Order ID not found.';
        }
    });

```

```

});
// Registration
registerForm.addEventListener('submit', e => {
    e.preventDefault();
    const fullname =
document.getElementById('fullname').value.trim();
    const email =
document.getElementById('emailReg').value.trim().toLowerCase();

    const password = document.getElementById('passwordReg').value;
    if(!fullname || !email || !password) {
        alert('Please fill all registration fields.');
```

```

        return;
    }
    // Check if user exists
    if(localStorage.getItem(user_`${email}`)) {
        alert('User with this email already exists.');
```

```

        return;
    }
    const userObj = { fullname, email, password };
    localStorage.setItem(user_`${email}`, JSON.stringify(userObj));
    alert('Account created successfully! Please login.');
```

```

    registerForm.reset();
});
// Login
loginFormAuth.addEventListener('submit', e => {
    e.preventDefault();
    const email =
document.getElementById('emailLogin').value.trim().toLowerCase();
    const password =
document.getElementById('passwordLogin').value;
    const userStr = localStorage.getItem(user_`${email}`);
    if(!userStr) {
        alert('No account found with this email.');
```

```

        return;
    }
    const userObj = JSON.parse(userStr);
    if(userObj.password !== password) {
        alert('Incorrect password.');
```

```

        return;
    }

```

```

        currentUser = userObj;
        authContainer.style.display = 'none';
        appSection.style.display = 'block';
        userNameDisplay.textContent = currentUser.fullname;
loadCart();
loadOrders();
    renderMedicines();
    updateCart();
});
// Logout
logoutBtn.addEventListener('click', () => {
    currentUser = null;
    cart = {};
    authContainer.style.display = 'block';
    appSection.style.display = 'none';
    cartCountEl.textContent = '0';
    qrCodeContainer.innerHTML = '';
});
// Initialize app for first time or logged out
function initApp() {
    authContainer.style.display = 'block';
    appSection.style.display = 'none';
}
initApp();
</script>
</body>
</html>

```

APPENDICES-II

output

Create Account

Full Name:

Email:

Password:

Create Account

Login

Email:

Password:

Login



Welcome, naresh!

[Logout](#)

Available Medicines

Paracetamol 500
mg / 650 mg

₹30

[Add](#)Ibuprofen 200 mg
/ 400 mg

₹100

[Add](#)Amoxicillin 250
mg / 500 mg

₹120

[Add](#)

Cetirizine 10 mg

₹39

[Add](#)Himalaya
Ashvagandha
General
Wellness, 60
Tablets

₹240

[Add](#)Omeprazole – 20
mg or 40 mg

₹50

[Add](#)

Aspirin – 75 mg

₹39

[Add](#)Pantoprazole –
40 mg

₹25

[Add](#)Salbutamol
Syrup - 2 mg/5 ml

₹150

[Add](#)Domperidone
Syrup-5 mg/5 ml

₹120

[Add](#)

Your Cart

Dettol Lime Fresh Disinfectant Liquid 200 ml ₹149

Pedigree Chicken & Milk Dry Food for Puppy Dog 2.8 kg ₹650

Lifebuoy Total Hand Sanitizer 50 ml ₹50

Total: ₹849



Place Your Order

Payment Method:

Cash on Delivery ▼

Full Name:

naresh

Phone Number:

8666885643

Pincode:

534201

Delivery Address:

Bhimavaram,DNR college Attached Boy's Hostel

Place Order

Track Your Order



Enter your Order ID:

ORD963650

Track

Order Status: Order Placed